Table of Contents

1. Introduction
    1.1. Scope and Assumptions
2. How To Read This Document
3. Document Health Snapshot
4. Terms and Definitions
    4.1. Diagram Color and Shape Legend
5. Architecture Intent View
    5.1. What This View Answers
    5.2. System Boundary Diagram
    5.3. System Decomposition Diagram
    5.4. Functional Group Manhattan Matrix
    5.5. Functional Unit Dependency Diagram
    5.6. Functional Groups (View Scoped)
    5.7. Functional Units (View Scoped)
6. Communication View
    6.1. What This View Answers
    6.2. Coverage Gaps
    6.3. Recommended Next Evidence Additions
    6.4. Communication Path Graph
    6.5. Functional Groups (View Scoped)
    6.6. Functional Units (View Scoped)
7. Deployment View
    7.1. What This View Answers
    7.2. Coverage Gaps
    7.3. Recommended Next Evidence Additions
    7.4. Deployment Artifact Relationship Graph
    7.5. Platform Operations Graph
    7.6. Functional Groups (View Scoped)
    7.7. Functional Units (View Scoped)
8. Security View
    8.1. What This View Answers
    8.2. Coverage Gaps
    8.3. Recommended Next Evidence Additions
    8.4. Attack Path Diagram
    8.5. Security Logging and Observability
    8.6. Functional Groups (View Scoped)
    8.7. Functional Units (View Scoped)
9. Traceability View
    9.1. What This View Answers
    9.2. Coverage Gaps
    9.3. Recommended Next Evidence Additions
    9.4. Traceability Evidence Graph
    9.5. Requirement-to-Unit Mapping

9.6. Requirement-to-Unit-to-Evidence Coverage
10. Traceability Appendix
10.1. Requirement Details
10.2. Verification Inventory
10.3. Verification Result Mapping
11. Reference Index
11.1. Authored References
11.2. Catalog References
11.3. Inferred Runtime References
11.4. Inferred Code References
11.5. Verification References

---

1. Introduction

This document describes the payments architecture using authored functional design and inferred runtime/code realization. It is intended to help product, platform, security, and implementation engineers reason about one shared model. Functional design is kept stable while realization details are inferred from infrastructure and source artifacts.

1.1. Scope and Assumptions

- Functional Groups and Functional Units are the stable authored design backbone.
- Runtime and code realization are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- Requirement alignment and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
3. Traceability View: requirement mapping, evidence coverage, and confidence.
4. Reference Appendix: supporting reference IDs and inferred-source links.

3. Document Health Snapshot

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Architecture Intent View (`VIEW-ARCHITECTURE-INTENT`)	stable	Stable authored capability boundaries and ownership split; changes are intentional and infrequent.	6	6	0	high	Defaults authored units to covered; use authoredStatus as the primary publication signal.
Communication View (`VIEW-COMMUNICATION`)	in-review	Communication semantics are partially inferred and still being refined with explicit protocol and interaction metadata.	4	3	1	medium	Counts a unit as covered when direct inferred runtime evidence exists.
Deployment View (`VIEW-DEPLOYMENT`)	in-review	Deployment chain and control-plane mapping are mostly present, but ownership evidence remains incomplete for some units.	6	5	1	high	Counts a unit as covered when direct inferred runtime evidence exists.
Security View (`VIEW-SECURITY`)	draft	Threat mapping is present for key vectors, but control expectations and coverage breadth are still evolving.	6	4	2	medium	Counts a unit as covered when explicit authored attack-vector mapping exists.
Traceability View (`VIEW-TRACEABILITY`)	in-review	Requirement-to-unit linkage is present; cross-layer ownership consistency still needs tightening.	6	5	4	high	Counts a unit as covered when direct inferred runtime or code evidence exists.

4. Terms and Definitions

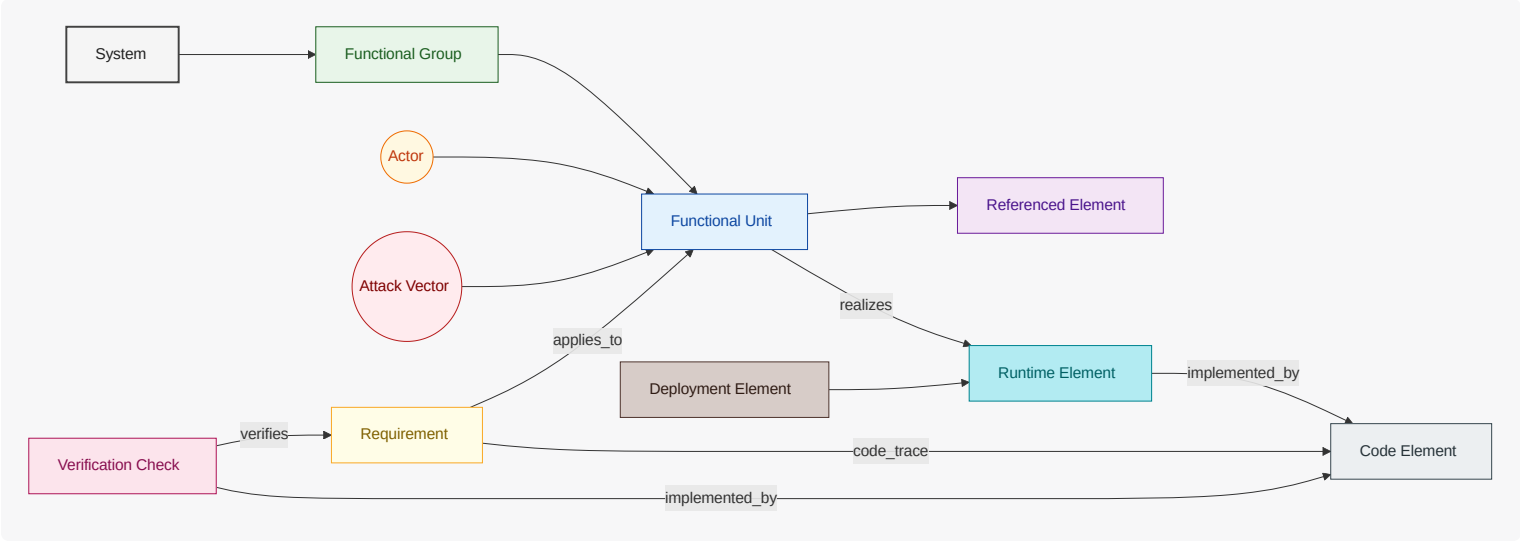
Term	Definition
3DS verification is required (COND-3DS-REQUIRED)	Condition indicating strong customer authentication must be performed. Aliases:3DS required
actor (EM-ACTOR)	A person or operational role that interacts with functional units.
attack vector (EM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
authored mapping (EM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.
bank gateway endpoint (REF-BANK-GATEWAY-ENDPOINT)	External banking endpoint used for card authorization and decline decisions. Aliases:bank API
bank link is available (STATE-BANK-LINK-UP)	Connectivity state showing bank gateway communication is currently healthy. Aliases:bank link up
bank link is unavailable (EVT-BANK-LINK-UNAVAILABLE)	Event raised when payment engine cannot reach the external bank gateway. Aliases:bank link down
checkout handling (FU-CHECKOUT)	Handles checkout session start and customer-facing payment messaging. Aliases:checkout
checkout is active (STATE-CHECKOUT-ACTIVE)	Processing state where the user is actively submitting or confirming payment. Aliases:checkout active
cluster provisioning (FU-CLUSTER-PROVISIONING)	Provides cluster-level platform setup and environment bootstrap. Aliases:day1 provisioning
code element (EM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
compromised dependency (AV-COMPROMISED-DEPENDENCY)	Supply-chain compromise in runtime image or code dependency. Aliases:dependency compromise
customer (ACT-CUSTOMER)	End user submitting payment details during checkout. Aliases:buyer
decline reason (DATA-DECLINE-REASON)	Structured reason code or message explaining why authorization failed. Aliases:decline code
fraud check is enabled (FEAT-FRAUD-CHECK)	Feature flag controlling whether risk scoring is required before authorization. Aliases:fraud check
fraud evaluation (FG-FRAUD)	Functional group for risk scoring and fraud review capabilities. Aliases:fraud detection system, fraud domain

Term	Definition
fraudulent transaction pattern (AV-FRAUDULENT-TRANSACTION-PATTERN)	Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals. Aliases:fraud abuse, payment fraud
functional group (EM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
functional unit (EM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.
gitops operations (FU-GITOPS-OPERATIONS)	Operates declarative delivery and release management flows. Aliases:day2 operations
helm platform (REF-HELM-PLATFORM)	Runtime packaging and deployment substrate for workloads. Aliases:helm
inferred mapping (EM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
malicious api request (AV-MALICIOUS-API-REQUEST)	Crafted payload attempting to bypass validation or authorization logic. Aliases:malicious request
normal mode is active (MODE-NORMAL)	Standard operating mode with full external dependency usage enabled. Aliases:normal mode
payment authorization (FU-PAYMENT-AUTHORIZATION)	Coordinates authorization flow and bank gateway interaction. Aliases:auth
payment authorization is requested (EVT-PAYMENT-AUTH-REQUESTED)	Event raised when checkout asks payment engine to authorize a transaction. Aliases:payment auth requested
payment id (DATA-PAYMENT-ID)	Stable identifier used to correlate a payment across services and records.
payment is declined (EVT-PAYMENT-DECLINED)	Event indicating authorization was rejected and customer decline handling must execute. Aliases:payment declined
payment review is pending (STATE-REVIEW-PENDING)	Transaction state requiring manual support review before final disposition. Aliases:review pending
payments (FG-PAYMENTS)	Functional group for checkout and authorization capabilities. Aliases:payments domain, payments system
platform (FG-PLATFORM)	Functional group for provisioning and platform operations capabilities. Aliases:platform engineering
platform operator (ACT-PLATFORM-OPERATOR)	Role responsible for GitOps and platform lifecycle operations. Aliases:platform ops

Term	Definition
postgresql client driver (REF-POSTGRES-DRIVER)	Third-party library used for application data persistence connectivity. Aliases:postgres driver
referenced element (EM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
replayed authorization callback (AV-REPLAYED-AUTH-CALLBACK)	Replay of a valid callback to induce duplicate or unauthorized state transitions. Aliases:callback replay
risk score (DATA-RISK-SCORE)	Numeric fraud score produced by risk evaluation logic.
risk score is high (EVT-RISK-HIGH)	Event emitted when computed fraud risk exceeds the configured high-risk threshold. Aliases:high risk
risk scoring (FU-RISK-SCORING)	Evaluates transaction risk and returns score classification. Aliases:fraud scoring
risk threshold is exceeded (COND-RISK-THRESHOLD-EXCEEDED)	Condition indicating risk score crossed a policy threshold for escalation. Aliases:risk above threshold
runtime element (EM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
support agent (ACT-SUPPORT)	Operations role that investigates declines and high-risk review cases. Aliases:support
support agent is notified (EVT-SUPPORT-NOTIFIED)	Event indicating manual review notification was sent to support operations. Aliases:support notified
support review (FU-SUPPORT-REVIEW)	Supports manual review and escalation handling for risky transactions. Aliases:manual review
upward linking (EM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

4.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



5. Architecture Intent View

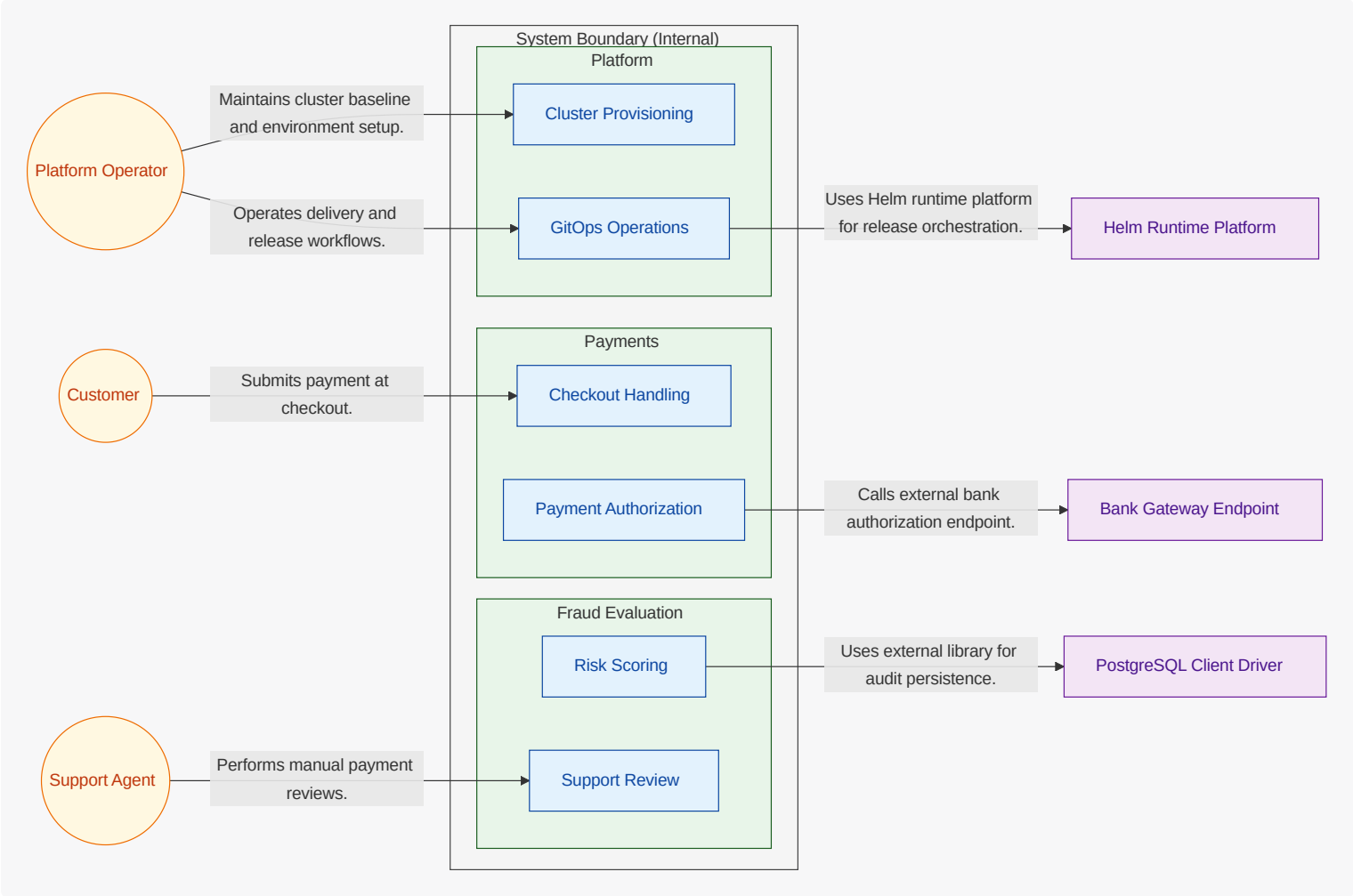
Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

5.1. What This View Answers

- What are the main capability areas?
- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

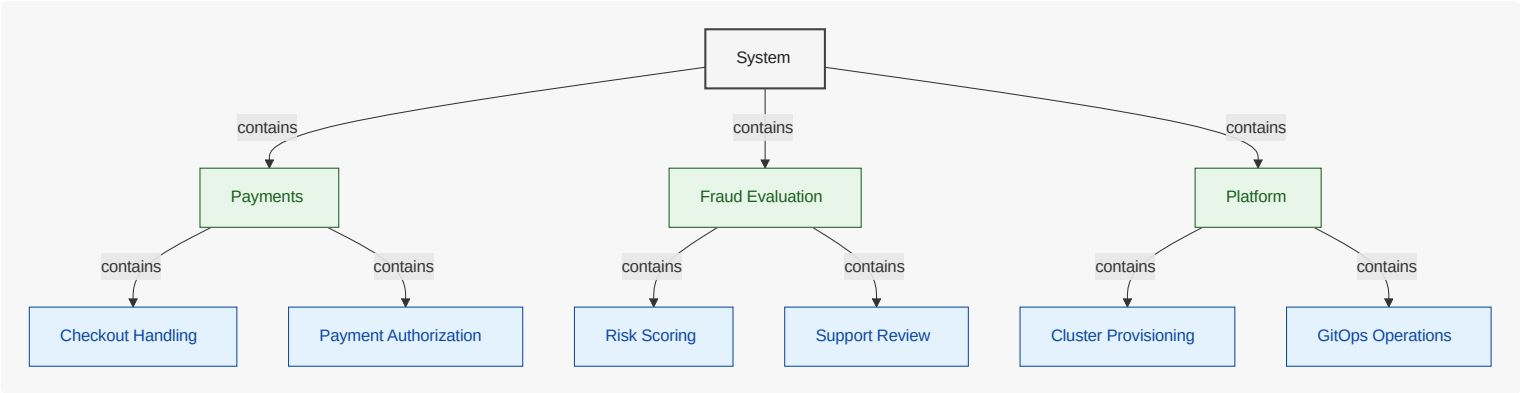
5.2. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the `System Boundary (Internal)` container, while actors and referenced dependencies are shown outside the boundary.



5.3. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.

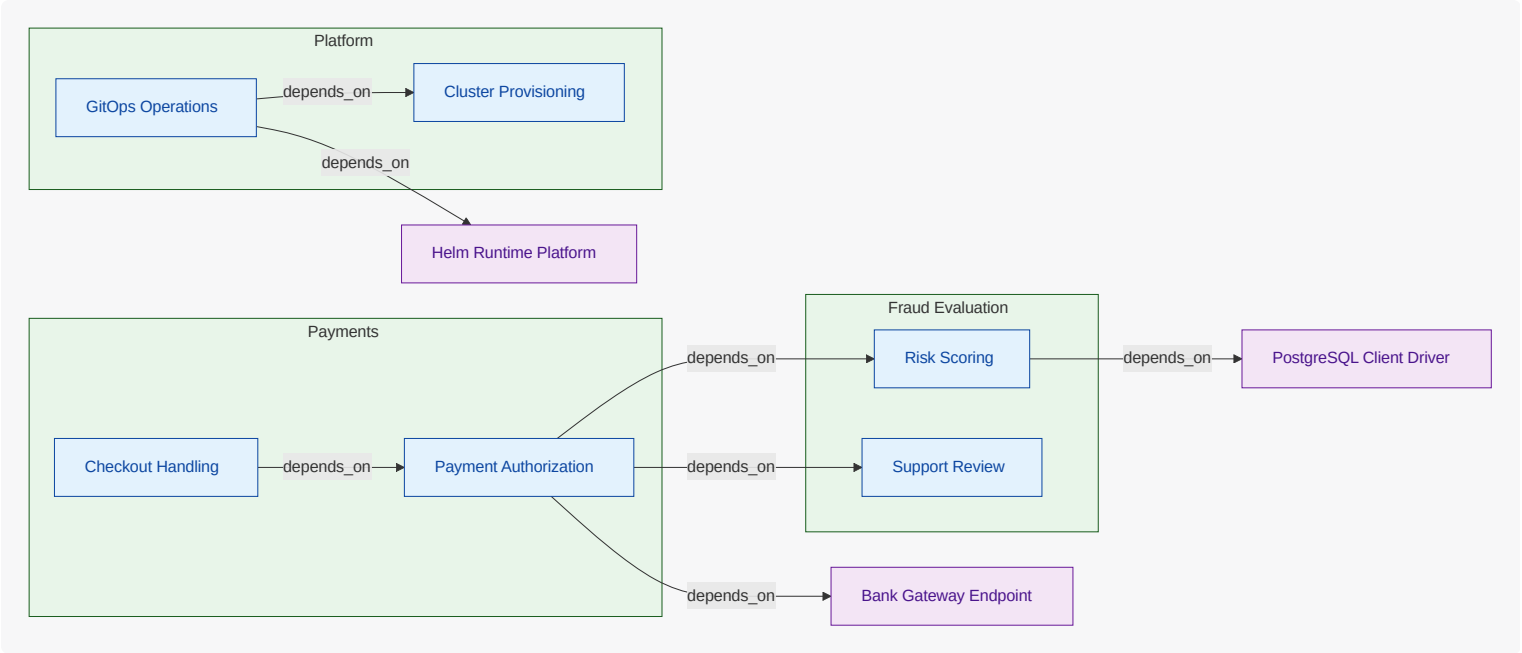


5.4. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

5.5. Functional Unit Dependency Diagram

What this diagram shows: dependency paths between functional units (grouped in Functional Group containers) and key referenced elements. Arrow meaning: `A -→|depends_on| B` means unit `A` requires behavior or data from `B` to fulfill its responsibility.



5.6. Functional Groups (View Scoped)

5.6.1. Payments (FG-PAYMENTS)

Description: Core payment checkout and authorization domain.

The payments group defines the product-facing payment journey from user intent to final payment outcome. It owns the consistency of customer messaging and transaction decisions when a payment is attempted, declined, retried, or temporarily unavailable. The group keeps customer flow stable while depending on fraud and external bank capabilities.

Payments Architecture Intent Design

payments defines the business capability that turns customer intent into a clear payment outcome. It sets the responsibilities between checkout handling and payment authorization, and defines where approval, decline, and fallback decisions are made.

5.6.2. Fraud Evaluation (FG-FRAUD)

Description: Risk scoring and fraud audit domain.

The fraud evaluation group exists to reduce unsafe approvals while preserving throughput for legitimate payments. It combines automated risk scoring with manual support review so uncertain transactions move through a controlled escalation path. The group ensures risk decisions are explainable, auditable, and operationally actionable.

Fraud Evaluation Architecture Intent Design

fraud evaluation defines how risk evidence is transformed into actionable payment decisions. It separates automated risk scoring from human support review so escalation paths are explicit, explainable, and policy-constrained.

5.6.3. Platform (FG-PLATFORM)

Description: Cluster provisioning and GitOps operations domain.

The platform group provides the runtime and deployment foundation used by all product capabilities. It owns environment provisioning, namespace and cluster boundaries, and continuous reconciliation behavior. The group enables safe delivery and reliable operations without owning payment business logic.

Platform Architecture Intent Design

platform provides the foundational capabilities required for all product workloads to run safely. It intentionally models provisioning and reconciliation as first-class architecture responsibilities, not background implementation detail.

5.7. Functional Units (View Scoped)

5.7.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk scoring computes and classifies transaction risk before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior.

Risk Scoring Architecture Intent Design

risk scoring produces risk classifications that are actionable for authorization policy. Functional design requires predictable threshold behavior and explainable outputs for operators.

Purpose: Risk scoring computes and classifies transaction risk before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior. Owned responsibility: functional responsibility for risk scoring; includes decision and orchestration flow to PostgreSQL Client Driver; primary requirement scope: REQ-PAY-002, REQ-PAY-005 Key collaborators: PostgreSQL Client Driver Owned decisions: REQ-PAY-002, REQ-PAY-005 Key inputs and outputs: consumes Payment Authorization; produces depends_on → PostgreSQL Client Driver Non-goals / boundary notes: this unit boundary is intentionally scoped to authored responsibilities and explicit collaborator contracts.

5.7.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support review handles manual decisions for escalated or ambiguous payment cases. It gives support operators context to approve, reject, or request additional verification in a controlled flow. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable.

Support Review Architecture Intent Design

support review provides controlled human decision flow for escalated transactions. Functional design ensures review actions are policy-bounded and outcome transitions are explicit.

Purpose: Support review handles manual decisions for escalated or ambiguous payment cases. It gives support operators context to approve, reject, or request additional verification in a controlled flow. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable. Owned responsibility: functional responsibility for support review; primary requirement scope: REQ-PAY-004 Key collaborators: none Owned decisions: REQ-PAY-004 Key inputs and outputs: consumes Payment Authorization; produces derived unit outcomes from authored mappings Non-goals / boundary notes: this unit boundary is intentionally scoped to authored responsibilities and explicit collaborator contracts.

5.7.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout handling is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear customer feedback for each outcome. It delegates decision logic to payment authorization while protecting the user experience boundary.

Checkout Handling Architecture Intent Design

checkout handling owns customer-facing request normalization and response semantics. Functional design ensures the customer receives a coherent outcome regardless of approval, decline, or temporary dependency failure conditions.

Purpose: Checkout handling is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear customer feedback for each outcome. It delegates decision logic to payment authorization while protecting the user experience boundary. Owned responsibility: functional responsibility for checkout handling; includes decision and orchestration flow to Payment Authorization; primary requirement scope: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Key collaborators: Payment Authorization Owned decisions: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Key inputs and outputs: consumes none; produces depends_on → Payment Authorization Non-goals / boundary notes: this unit boundary is intentionally scoped to authored responsibilities and explicit collaborator contracts.

5.7.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment authorization orchestrates the final transaction decision path. It coordinates fraud scoring, external bank interactions, and escalation to support review when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable.

Payment Authorization Architecture Intent Design

payment authorization defines the decision policy orchestration for each transaction. It combines fraud input, bank outcomes, and escalation rules into one deterministic business decision.

Purpose: Payment authorization orchestrates the final transaction decision path. It coordinates fraud scoring, external bank interactions, and escalation to support review when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable. Owned responsibility: functional responsibility for payment authorization; includes decision and orchestration flow to Bank Gateway Endpoint and decision and orchestration flow to Risk Scoring; primary requirement scope: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Key collaborators: Bank Gateway Endpoint, Risk Scoring, Support Review Owned decisions: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Key inputs and outputs: consumes Checkout Handling; produces depends_on → Bank Gateway Endpoint; depends_on → Risk Scoring; depends_on → Support Review Non-goals / boundary notes: this unit boundary is intentionally scoped to authored responsibilities and explicit collaborator contracts.

5.7.5. Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Group: FG-PLATFORM

Cluster provisioning creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations.

Cluster Provisioning Architecture Intent Design

cluster provisioning defines the repeatable environment baseline for all runtime workloads. Functional design establishes deterministic creation of cluster and namespace structure.

Purpose: Cluster provisioning creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations. Owned responsibility: functional responsibility for cluster provisioning Key collaborators: none Owned decisions: no explicit authored requirement trigger Key inputs and outputs: consumes GitOps Operations; produces derived unit outcomes from authored mappings Non-goals / boundary notes: this unit boundary is intentionally scoped to authored responsibilities and explicit collaborator contracts.

5.7.6. GitOps Operations (FU-GITOPS-OPERATIONS)

Group: FG-PLATFORM

GitOps operations continuously reconciles intended release state with actual runtime state. It governs rollout flow, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability.

GitOps Operations Architecture Intent Design

gitops operations defines how declared release intent becomes enforced runtime state. Functional design ensures delivery behavior is predictable and drift is corrected consistently.

Purpose: GitOps operations continuously reconciles intended release state with actual runtime state. It governs rollout flow, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability. Owned responsibility: functional responsibility for gitops operations; includes decision and orchestration flow to Cluster Provisioning and decision and orchestration flow to Helm Runtime Platform Key collaborators: Cluster Provisioning, Helm Runtime Platform Owned decisions: no explicit authored requirement trigger Key inputs and outputs: consumes none; produces depends_on → Cluster Provisioning; depends_on → Helm Runtime Platform Non-goals / boundary notes: this unit boundary is intentionally scoped to authored responsibilities and explicit collaborator contracts.

6. Communication View

Show who communicates with whom across key request paths, including interface edges and dependency handoffs.

6.1. What This View Answers

- Who talks to whom and over which interfaces?
- Which edges are synchronous, asynchronous, event-driven, or callback-oriented?
- Which communication paths are critical to the main request flow?

6.2. Coverage Gaps

Coverage summary: 3/4 units have direct evidence coverage in this view.

- Support Review has no direct inferred runtime/deployment evidence yet.

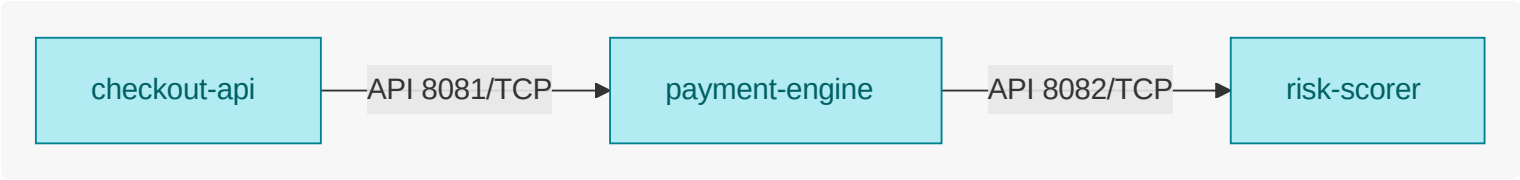
6.3. Recommended Next Evidence Additions

- Add explicit communication metadata (protocol, sync/async/event/callback) to authored mappings where currently implicit.
- Expose service protocol/port metadata in Helm or manifest values to enrich interface-edge inference.
- Capture missing communicating runtime units by adding discoverable workload artifacts under scanned inference roots.

6.4. Communication Path Graph

What this diagram shows: inferred unit-to-unit request/call flow, including available interface/port labels.

> **NOTE** This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or interface details indicate missing derived communication evidence, not necessarily missing behavior.



6.5. Functional Groups (View Scoped)

6.5.1. Payments (FG-PAYMENTS)

Payments Communication Design

payments runs the public request path and keeps transaction context intact across runtime hops. The runtime design emphasizes low latency, deterministic request sequencing, and clear operational signals when dependencies slow down or fail.

6.5.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Communication Design

fraud evaluation runs as a critical dependency on the authorization path. Runtime behavior must return stable contracts quickly while preserving enough evidence for operator diagnosis and audit when high-risk classifications occur.

6.6. Functional Units (View Scoped)

6.6.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Communication Design

risk scoring runs as a synchronous dependency with strict response-shape stability. Runtime design favors deterministic outputs, bounded latency, and clear degradation semantics for callers.

Communication Aspect	View-Scoped Detail
Inbound interfaces	Payment Authorization
Outbound interfaces	depends_on → PostgreSQL Client Driver
Messages/events consumed	REQ-PAY-002, REQ-PAY-005
Messages/events produced	depends_on → PostgreSQL Client Driver
Retry/idempotency expectations	Not explicitly modeled in authored mappings; define per interface contract as needed.
Timeout/failure behavior	PostgreSQL Client Driver
Communication evidence	runtime: risk, risk/risk-scorer

6.6.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Communication Design

support review runtime optimizes for operator clarity and decision traceability over throughput. Runtime design emphasizes state visibility, reversible transitions where allowed, and reliable action logging.

Communication Aspect	View-Scoped Detail
Inbound interfaces	Payment Authorization
Outbound interfaces	derived unit outcomes from authored mappings
Messages/events consumed	REQ-PAY-004
Messages/events produced	derived unit outcomes from authored mappings
Retry/idempotency expectations	Not explicitly modeled in authored mappings; define per interface contract as needed.
Timeout/failure behavior	none
Communication evidence	authored unit with no direct derived runtime/code evidence yet

6.6.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Communication Design

checkout handling is the ingress runtime boundary and forwards normalized requests downstream. Runtime design emphasizes stable request contracts, context propagation, and graceful behavior under burst traffic and partial downstream degradation.

Communication Aspect	View-Scoped Detail
Inbound interfaces	none
Outbound interfaces	depends_on → Payment Authorization
Messages/events consumed	REQ-PAY-001, REQ-PAY-003, REQ-PAY-006
Messages/events produced	depends_on → Payment Authorization
Retry/idempotency expectations	Not explicitly modeled in authored mappings; define per interface contract as needed.
Timeout/failure behavior	Payment Authorization
Communication evidence	runtime: payments, payments/checkout-api

6.6.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Communication Design

payment authorization coordinates critical runtime dependencies and enforces decision-order determinism. Runtime design prioritizes idempotent transitions, explicit branch visibility, and controlled error normalization.

Communication Aspect	View-Scoped Detail
Inbound interfaces	Checkout Handling
Outbound interfaces	depends_on → Bank Gateway Endpoint; depends_on → Risk Scoring; depends_on → Support Review
Messages/events consumed	REQ-PAY-001, REQ-PAY-004, REQ-PAY-006
Messages/events produced	depends_on → Bank Gateway Endpoint; depends_on → Risk Scoring; depends_on → Support Review
Retry/idempotency expectations	Not explicitly modeled in authored mappings; define per interface contract as needed.
Timeout/failure behavior	Bank Gateway Endpoint, Risk Scoring, Support Review

Communication Aspect	View-Scoped Detail
Communication evidence	runtime: payments/payment-engine

7. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This view emphasizes deployment relationships and platform operations.

7.1. What This View Answers

- How does authored intent become deployed runtime?
- What control-plane objects manage deployable units?
- Where are platform ownership boundaries and missing ownership signals?

7.2. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view.

- Support Review has no direct inferred runtime/deployment evidence yet.

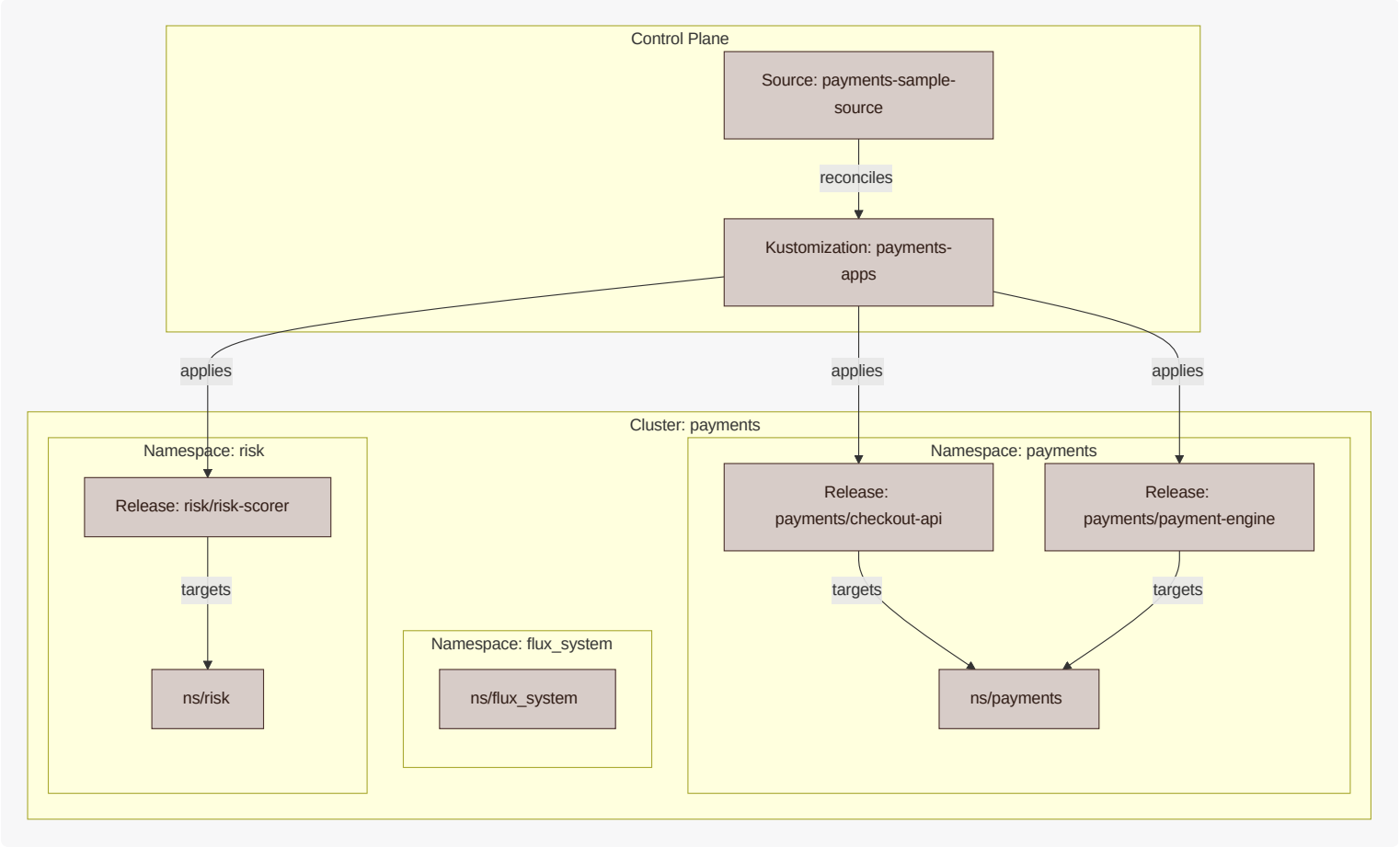
7.3. Recommended Next Evidence Additions

- Add owner metadata to Flux/Kustomization/Helm objects for clearer platform ownership mapping.
- Ensure deployment control artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking.

7.4. Deployment Artifact Relationship Graph

What this diagram shows: how source/control-plane artifacts connect to releases, namespaces, workloads, and target cluster context.

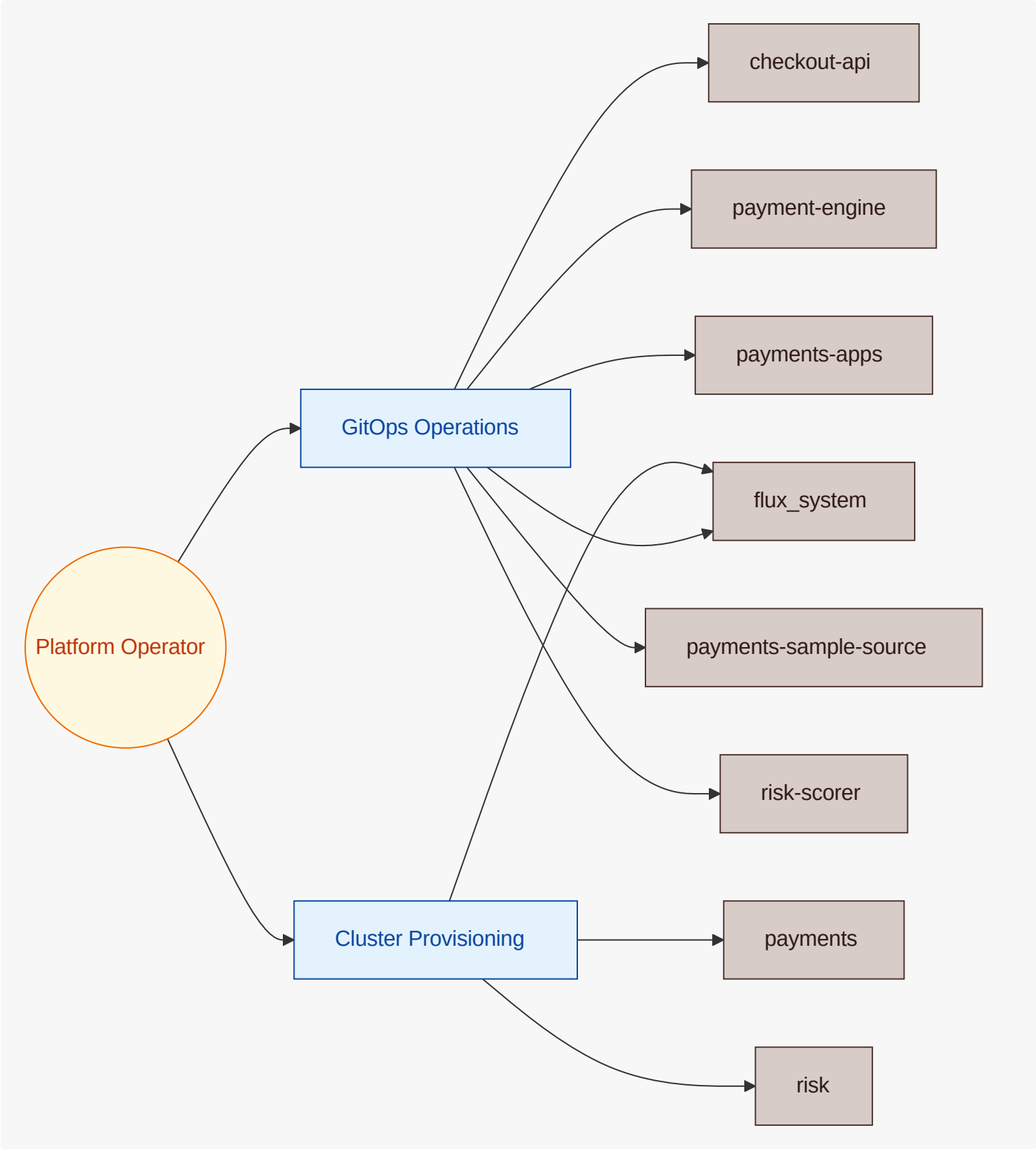
NOTE	This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete. NOTE: Namespace and cluster boundaries are rendered as containers when target/placement relationships are inferred. NOTE: Node labels include role hints (for example Source, Kustomization, Release, Workload, ns/…) to make similarly named elements easier to distinguish.



7.5. Platform Operations Graph

What this diagram shows: which platform actors operate which platform units and the control-plane targets they influence.

> **NOTE** Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.



7.6. Functional Groups (View Scoped)

7.6.1. Payments (FG-PAYMENTS)

Payments Deployment Design

payments is deployed so checkout and authorization can roll independently with minimal blast radius. Deployment design prioritizes safe promotion, fast rollback, and dependency readiness checks before policy-impacting releases are exposed.

7.6.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Deployment Design

fraud evaluation deployment is tuned for cautious rollout because scoring changes can alter approval rates. Deployment design favors staged promotions and rapid monitoring of scoring outcomes before wider exposure.

7.6.3. Platform (FG-PLATFORM)

Platform Deployment Design

platform deployment design governs how infrastructure intent is applied and kept in sync over time. It prioritizes deterministic sequencing, drift control, and recoverable change paths before application teams depend on new platform states.

7.7. Functional Units (View Scoped)

7.7.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Deployment Design

risk scoring deployment uses staged rollout because scoring changes have immediate decision impact. Deployment design includes observation windows for score distribution and escalation-rate changes.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-002, REQ-PAY-005
Deployment dependencies	PostgreSQL Client Driver
Inferred deployment evidence	runtime: risk, risk/risk-scorer

7.7.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Deployment Design

support review deployment isolates privileged workflows from public ingress components. Deployment design must preserve active review sessions across upgrades and ensure queue/review latency visibility.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-004
Deployment dependencies	none
Inferred deployment evidence	authored unit with no direct derived runtime/code evidence yet

7.7.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Deployment Design

checkout handling is released independently to protect user-facing velocity without destabilizing policy logic. Deployment design favors fast rollback and compatibility checks with the current authorization interface.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-001, REQ-PAY-003, REQ-PAY-006
Deployment dependencies	Payment Authorization
Inferred deployment evidence	runtime: payments, payments/checkout-api

7.7.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Deployment Design

payment authorization deployment is policy-sensitive and therefore rollout-cautious. Releases should stage gradually, verify dependency health, and preserve in-flight transaction continuity.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-001, REQ-PAY-004, REQ-PAY-006
Deployment dependencies	Bank Gateway Endpoint, Risk Scoring, Support Review
Inferred deployment evidence	runtime: payments/payment-engine

7.7.5. Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Group: FG-PLATFORM

Cluster Provisioning Deployment Design

cluster provisioning deployment is infrastructure-first and sequence-sensitive. Deployment design requires deterministic change plans, drift awareness, and rollback strategy before apply.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored requirement trigger
Deployment dependencies	none
Inferred deployment evidence	runtime: payments

7.7.6. GitOps Operations (FU-GITOPS-OPERATIONS)

Group: FG-PLATFORM

GitOps Operations Deployment Design

gitops operations deployment defines the release control plane for application artifacts. Deployment design prioritizes policy gates, controlled promotion, and resilient rollback mechanics.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored requirement trigger
Deployment dependencies	Cluster Provisioning, Helm Runtime Platform
Inferred deployment evidence	runtime: flux-system/payments-apps, flux-system/payments-sample-source, flux_system

8. Security View

Show inferred exposure and dependency risk points aligned to unit boundaries, focused on attack paths and security evidence.

8.1. What This View Answers

- What are the main trust boundaries and attack paths?
- Which units are exposed and what security evidence exists?
- Where is the threat model incomplete?

8.2. Coverage Gaps

Coverage summary: 4/6 units have direct evidence coverage in this view.

- Cluster Provisioning has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.
- GitOps Operations has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.

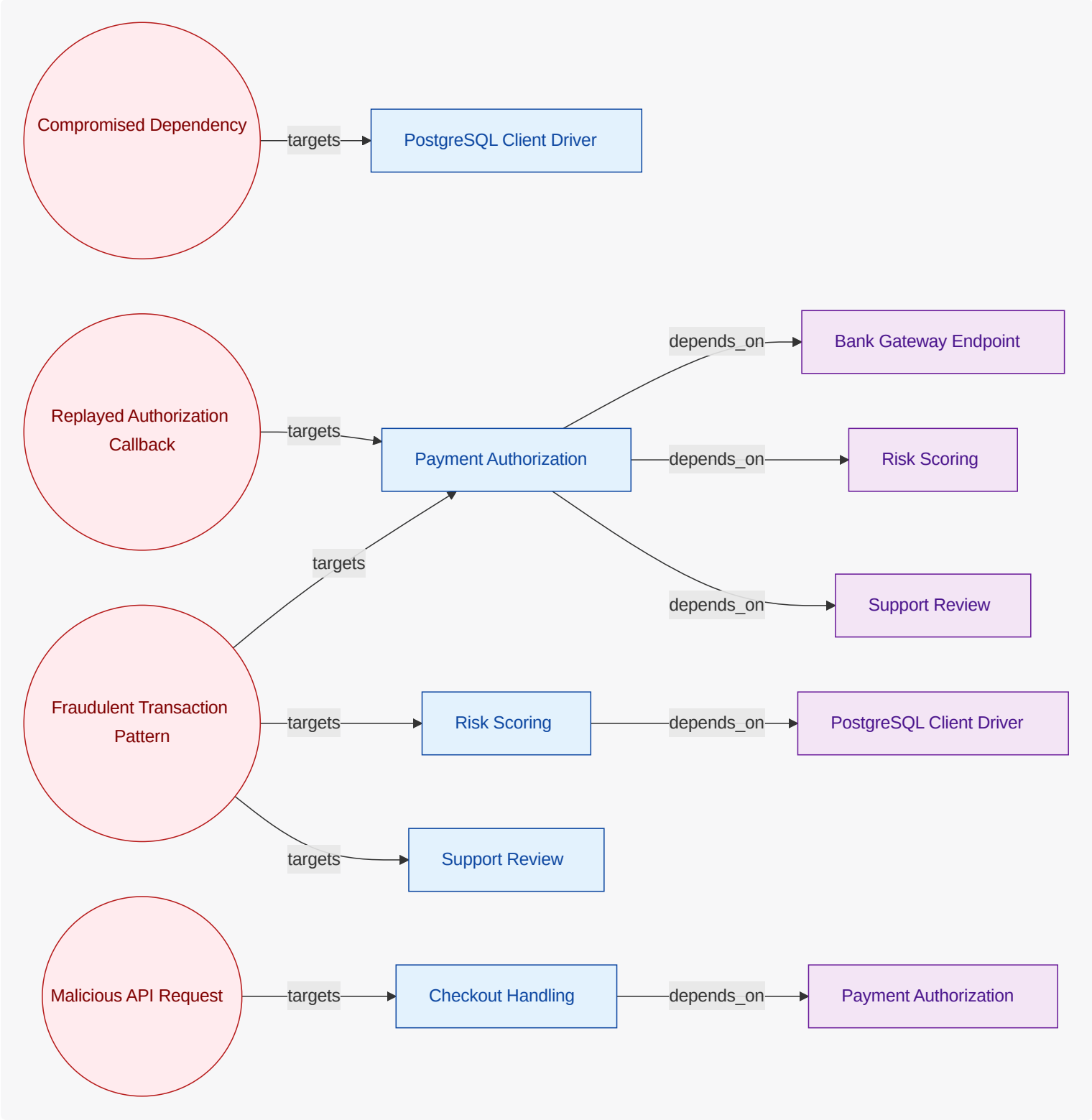
8.3. Recommended Next Evidence Additions

- Add explicit authored attack-vector mappings for units currently marked without threat linkage.
- Add security signal ownership metadata (logs/alerts/events) for unresolved observability evidence.
- Map additional security-relevant dependencies and exposures into authored mappings for traceable coverage.

8.4. Attack Path Diagram

What this diagram shows: authored attack vectors, their targeted units, and security-relevant dependency paths.

NOTE	Security relationships shown here are explicitly modeled or inferred from available evidence and may be incomplete. Absence of a path does not imply absence of risk. NOTE: Threats that are currently mitigated (including fraud/abuse threats) should still be modeled as attack vectors for auditability and control-trace visibility.



8.5. Security Logging and Observability

Signal	Layer	Owner	Evidence
application security telemetry hooks	code	FU-RISK-SCORING	code support/audit_envelope.ts

Signal	Layer	Owner	Evidence
fraud and abuse detection code paths	code	FU-RISK-SCORING	code risk_scorer.ts
deployment and platform audit events	deployment	FU-CHECKOUT	helmrelease checkout-api
deployment and platform audit events	deployment	FU-CHECKOUT	namespace payments
deployment and platform audit events	deployment	FU-CLUSTER-PROVISIONING	cluster payments
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	gitrepository payments-sample-source
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	kustomization payments-apps
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	namespace flux_system
deployment and platform audit events	deployment	FU-PAYMENT-AUTHORIZATION	helmrelease payment-engine
deployment and platform audit events	deployment	FU-RISK-SCORING	helmrelease risk-scorer
deployment and platform audit events	deployment	FU-RISK-SCORING	namespace risk
abuse and fraud decision logs	runtime	FU-RISK-SCORING	helmrelease risk-scorer
abuse and fraud decision logs	runtime	FU-RISK-SCORING	namespace risk

8.6. Functional Groups (View Scoped)

8.6.1. Payments (FG-PAYMENTS)

Payments Security Design

payments protects public trust boundaries where untrusted input and external callbacks enter the system. Security design focuses on strict validation, replay resistance, and safe failure behavior that preserves customer trust during degraded dependency conditions.

8.6.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Security Design

fraud evaluation security focuses on decision integrity and review integrity. The design assumes attempted manipulation of risk inputs, dependency trust, and manual review workflows, and therefore requires strong evidence lineage and tamper-evident operator actions.

8.6.3. Platform (FG-PLATFORM)

Platform Security Design

platform security design establishes baseline controls inherited by all higher-layer units. It focuses on trusted reconciliation, isolation defaults, and policy enforcement points where compromise would have broad systemic impact.

8.7. Functional Units (View Scoped)

8.7.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Security Design

risk scoring security assumes adversarial input attempts and model manipulation pressure. Design controls enforce strict input handling and evidence capture for post-event investigation.

Attack surface: Fraudulent Transaction Pattern Main threats: Fraudulent Transaction Pattern Expected controls: define control expectations in authored security narratives and mapped dependencies. Security-relevant dependencies: PostgreSQL Client Driver Security evidence present: runtime: risk, risk/risk-scorer | code modules: risk_scorer, support Security evidence missing: close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.7.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Security Design

support review security requires strong operator identity controls and tamper-evident audit trails. The design prevents silent overrides and guarantees every manual decision is attributable.

Attack surface: Fraudulent Transaction Pattern Main threats: Fraudulent Transaction Pattern Expected controls: define control expectations in authored security narratives and mapped dependencies. Security-relevant dependencies: none Security evidence present: authored unit with no direct derived runtime/code evidence yet Security evidence missing: close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.7.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Security Design

checkout handling security protects the internet-facing ingress. The design requires strict validation, abuse resistance, and traceable rejection behavior so malicious inputs are contained without harming legitimate customer flows.

Attack surface: Malicious API Request Main threats: Malicious API Request Expected controls: define control expectations in authored security narratives and mapped dependencies. Security-relevant dependencies: Payment Authorization Security evidence present: runtime: payments, payments/checkout-api | code modules: checkout_api Security evidence missing: close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.7.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Security Design

payment authorization security protects decision integrity and callback integrity. The design defends against replay, policy probing, and ambiguous error channels that could be abused.

Attack surface: Fraudulent Transaction Pattern, Replayed Authorization Callback Main threats: Fraudulent Transaction Pattern, Replayed Authorization Callback Expected controls: define control expectations in authored security narratives and mapped dependencies. Security-relevant dependencies: Bank Gateway Endpoint, Risk Scoring, Support Review Security evidence present: runtime: payments/payment-engine | code modules: domain, payment_engine Security evidence missing: close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.7.5. Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Group: FG-PLATFORM

Cluster Provisioning Security Design

cluster provisioning security sets inherited baseline hardening and isolation defaults. Design must detect and remediate security drift early because downstream controls depend on this layer.

Attack surface: no explicit attack vector targeting this unit Main threats: no explicit attack vector targeting this unit Expected controls: define control expectations in authored security narratives and mapped dependencies. Security-relevant dependencies: none Security evidence present: runtime: payments Security evidence missing: close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.7.6. GitOps Operations (FU-GITOPS-OPERATIONS)

Group: FG-PLATFORM

GitOps Operations Security Design

gitops operations security protects source trust and reconciliation authority. The design prevents unauthorized state changes and enforces policy before runtime apply occurs.

Attack surface: no explicit attack vector targeting this unit Main threats: no explicit attack vector targeting this unit Expected controls: define control expectations in authored security narratives and mapped dependencies. Security-relevant dependencies: Cluster Provisioning, Helm Runtime Platform Security evidence present: runtime: flux-system/payments-apps, flux-system/payments-sample-source, flux_system Security evidence missing: close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

9. Traceability View

Show requirement-to-unit-to-evidence traceability, including coverage confidence and explicit evidence gaps.

9.1. What This View Answers

- Which requirements map to which units?
- Which authored units have runtime/code evidence and which do not?
- Where are confidence or completeness gaps in trace coverage?

9.2. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view.

- Cluster Provisioning has no direct inferred code evidence yet.
- GitOps Operations has no direct inferred code evidence yet.
- Support Review has no direct inferred code evidence yet.
- Support Review has no direct inferred runtime evidence yet.

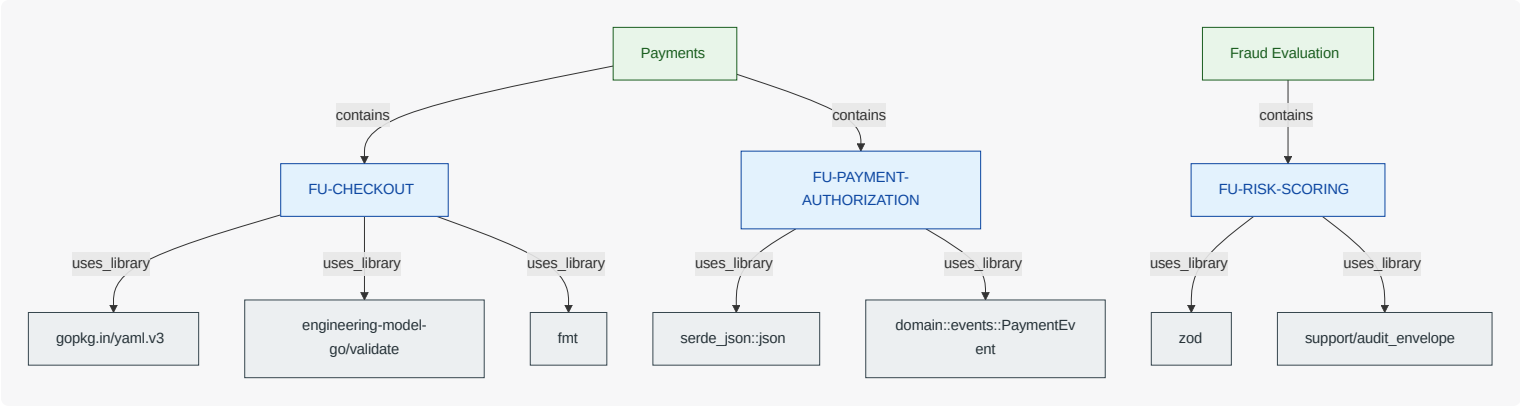
9.3. Recommended Next Evidence Additions

- Add explicit requirement-to-owner mappings where requirement scope is currently broad or ambiguous.
- Add ownership annotations for runtime and code artifacts where coverage is still unresolved.
- Prioritize closing gaps for units that are authored but lack both runtime and code evidence.

9.4. Traceability Evidence Graph

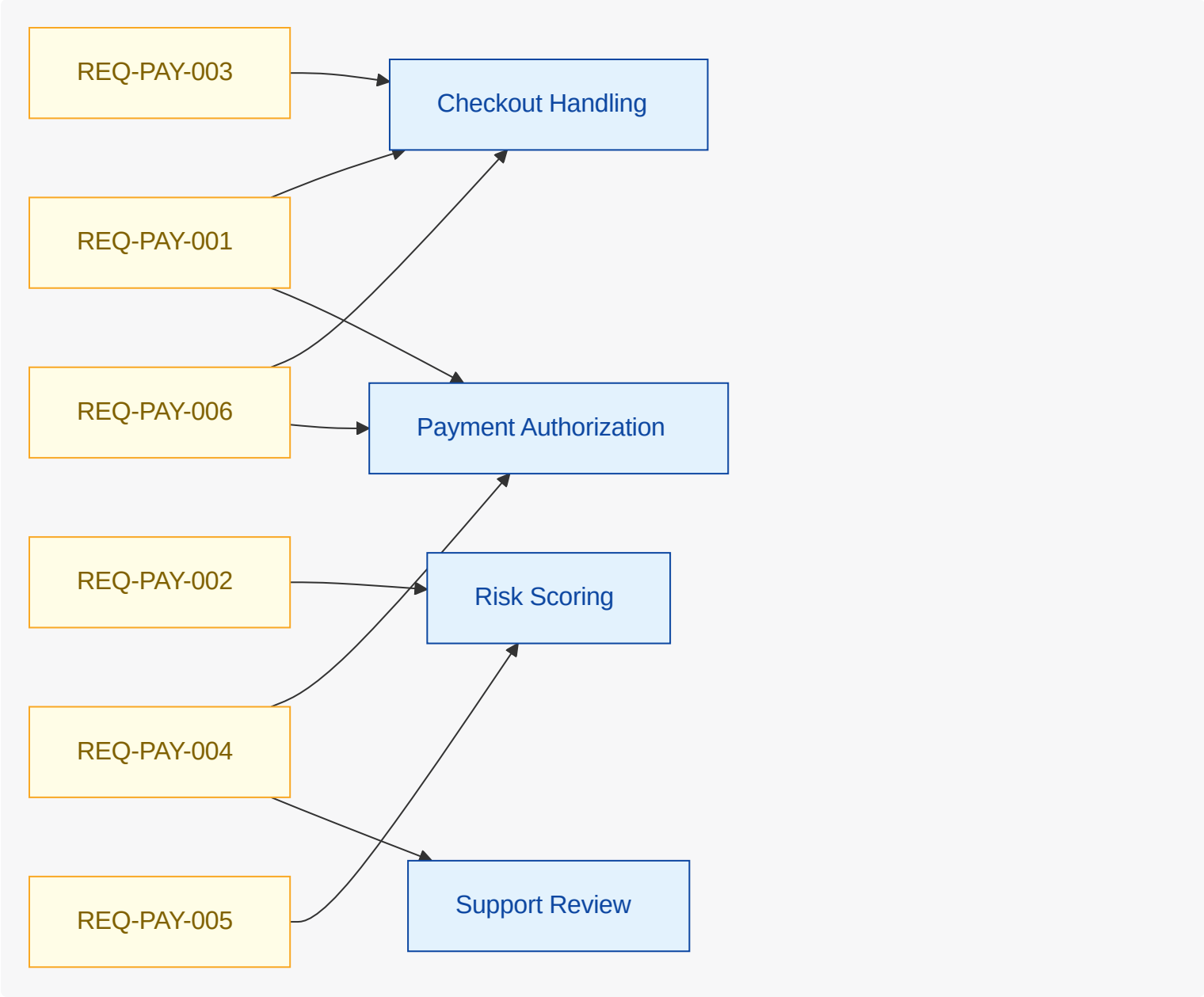
What this diagram shows: authored units and the inferred evidence links that support traceability confidence.

> **NOTE** This graph summarizes inferred ownership/evidence links and is intended for traceability confidence checks.



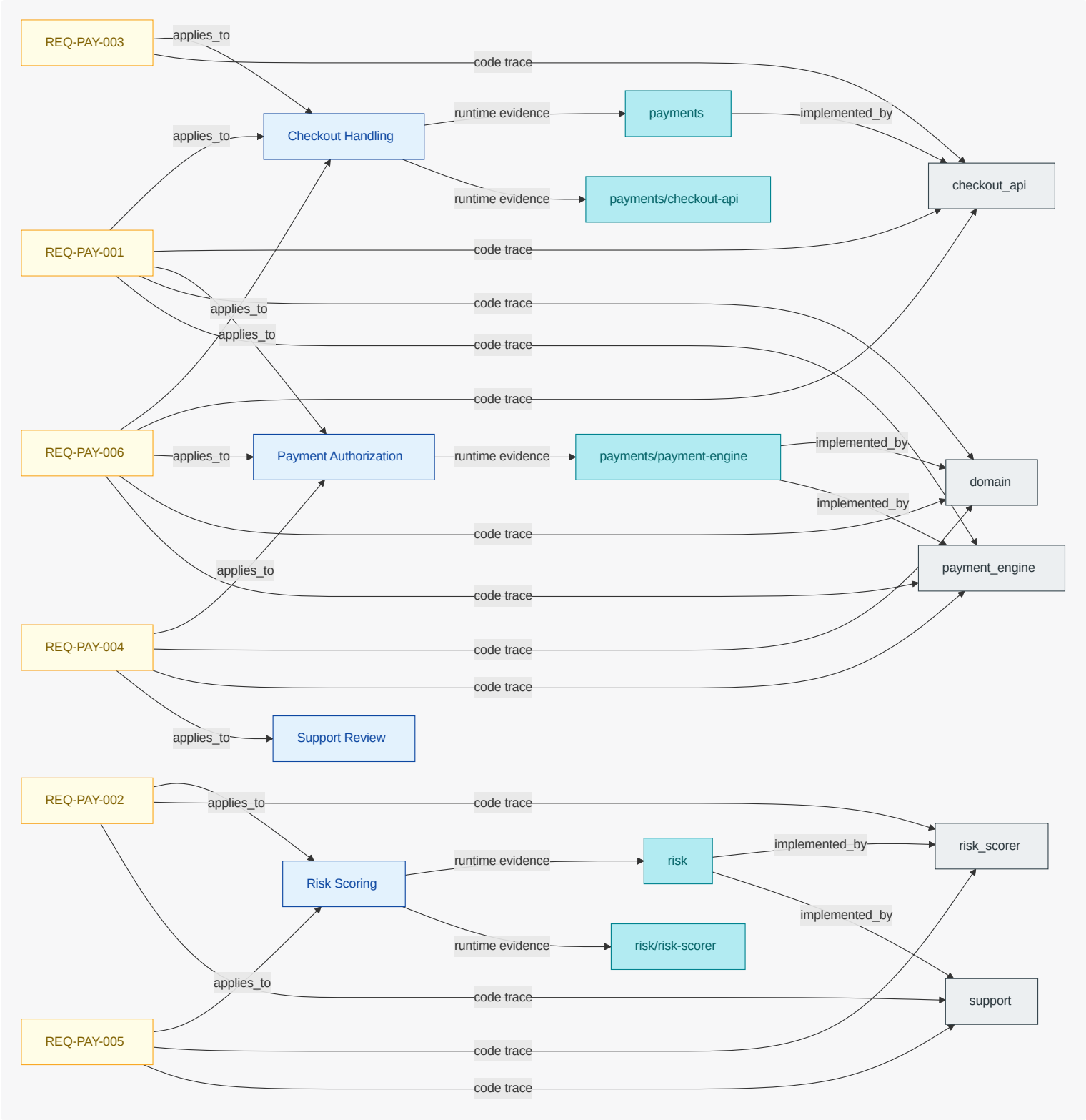
9.5. Requirement-to-Unit Mapping

What this diagram shows: direct authored mapping from requirements to the functional units they apply to.



9.6. Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: requirement-to-unit links extended with inferred runtime/code evidence attached to each unit.



Show requirement-to-unit mappings inferred from appliesTo and authored architecture ownership boundaries.

10. Traceability Appendix

10.1. Requirement Details

10.1.1. REQ-PAY-001

While checkout is active, when payment authorization is requested, the payments system shall create a payment session and request authorization from bank gateway. Notes: Core authorization path from checkout to bank gateway.

10.1.2. REQ-PAY-002

Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval. Notes: Fraud scoring gate before approval.

10.1.3. REQ-PAY-003

If payment is declined, then the payments system shall present a decline reason and a retry option to customer. Notes: Decline handling and customer feedback.

10.1.4. REQ-PAY-004

While checkout is active, when payment authorization is requested and risk score is high, the payments system shall place the payment in review pending state and notify support agent. Notes: Escalation flow for high-risk payments.

10.1.5. REQ-PAY-005

Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id and risk score. Notes: Auditability for authorization decisions.

10.1.6. REQ-PAY-006

If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer. Notes: Fallback behavior when external banking path is down.

10.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

10.2.1. Authorized Checkout Flow

Field	Value
ID	VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597
Kind	e2e
Status	not-run
Verifies Requirements	REQ-PAY-001
Test Code Elements	tests/e2e/authorized_checkout_flow.yaml

Field	Value
Derived Functional Owners	FU-CHECKOUT, FU-PAYMENT-AUTHORIZATION
Result Summary	not-run:1
Evidence	tests/e2e/authorized_checkout_flow.yaml
Notes	Inferred from test source artifact.

10.2.2. Audit Record Persistence Test

Field	Value
ID	VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2
Kind	integration
Status	partial
Verifies Requirements	REQ-PAY-005
Test Code Elements	tests/integration/audit_record_persistence_test.ts
Derived Functional Owners	FU-RISK-SCORING
Result Summary	partial:1
Evidence	test-results/audit-persistence-report.md, tests/integration/audit_record_persistence_test.ts
Notes	Inferred from test source artifact.

10.2.3. Fraud Gate Test

Field	Value
ID	VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD
Kind	integration
Status	pass
Verifies Requirements	REQ-PAY-002, REQ-PAY-004
Test Code Elements	tests/integration/fraud_gate_test.rs
Derived Functional Owners	FU-PAYMENT-AUTHORIZATION, FU-RISK-SCORING, FU-SUPPORT-REVIEW

Field	Value
Result Summary	pass:2
Evidence	test-results/integration-fraud-gate.json, tests/integration/fraud_gate_test.rs
Notes	Inferred from test source artifact.

10.2.4. Contract Checkout Decline

Field	Value
ID	VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26
Kind	test
Status	pass
Verifies Requirements	REQ-PAY-003, REQ-PAY-006
Test Code Elements	none
Derived Functional Owners	FU-CHECKOUT, FU-PAYMENT-AUTHORIZATION
Result Summary	pass:2
Evidence	test-results/contract-checkout-decline.json
Notes	Inferred from test result artifact.

10.3. Verification Result Mapping

What this section shows: per-requirement result mapping inferred from test-result artifacts.

Check ID	Requirement	Result	Evidence / Notes
VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597	REQ-PAY-001	not-run	none No parsed test-result artifact for this requirement yet.
VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD	REQ-PAY-002	pass	test-results/integration-fraud-gate.json
VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26	REQ-PAY-003	pass	test-results/contract-checkout-decline.json
VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD	REQ-PAY-004	pass	test-results/integration-fraud-gate.json

Check ID	Requirement	Result	Evidence / Notes
VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2	REQ-PAY-005	partial	test-results/audit-persistence-report.md
VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26	REQ-PAY-006	pass	test-results/contract-checkout-decline.json

11. Reference Index

11.1. Authored References

ACT-CUSTOMER

Kind: Actor Name: Customer

Starts checkout and confirms payment.

ACT-PLATFORM-OPERATOR

Kind: Actor Name: Platform Operator

Operates GitOps and platform lifecycle.

ACT-SUPPORT

Kind: Actor Name: Support Agent

Reviews high-risk and declined-payment cases.

AV-COMPROMISED-DEPENDENCY

Kind: Attack Vector Name: Compromised Dependency

Supply chain compromise in external SDK or image.

AV-FRAUDULENT-TRANSACTION-PATTERN

Kind: Attack Vector Name: Fraudulent Transaction Pattern

Coordinated abuse pattern using synthetic identities, stolen instruments, and velocity anomalies to seek unauthorized approvals.

AV-MALICIOUS-API-REQUEST

Kind: Attack Vector Name: Malicious API Request

Crafted payloads attempting to bypass authorization controls.

AV-REPLAYED-AUTH-CALLBACK

Kind: Attack Vector Name: Replayed Authorization Callback

Replay attack against external authorization callback flow.

FG-FRAUD

Kind: Functional Group Name: Fraud Evaluation

Risk scoring and fraud audit domain.

FG-PAYMENTS

Kind: Functional Group Name: Payments

Core payment checkout and authorization domain.

FG-PLATFORM

Kind: Functional Group Name: Platform

Cluster provisioning and GitOps operations domain.

FU-CHECKOUT

Kind: Functional Unit Name: Checkout Handling

Checkout handling is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear customer feedback for each outcome. It delegates decision logic to payment authorization while protecting the user experience boundary.

FU-CLUSTER-PROVISIONING

Kind: Functional Unit Name: Cluster Provisioning

Cluster provisioning creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations.

FU-GITOPS-OPERATIONS

Kind: Functional Unit Name: GitOps Operations

GitOps operations continuously reconciles intended release state with actual runtime state. It governs rollout flow, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability.

FU-PAYMENT-AUTHORIZATION

Kind: Functional Unit Name: Payment Authorization

Payment authorization orchestrates the final transaction decision path. It coordinates fraud scoring, external bank interactions, and escalation to support review when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable.

FU-RISK-SCORING

Kind: Functional Unit Name: Risk Scoring

Risk scoring computes and classifies transaction risk before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior.

FU-SUPPORT-REVIEW

Kind: Functional Unit Name: Support Review

Support review handles manual decisions for escalated or ambiguous payment cases. It gives support operators context to approve, reject, or request additional verification in a controlled flow. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable.

REF-BANK-GATEWAY-ENDPOINT

Kind: Referenced Element Name: Bank Gateway Endpoint

external_service_endpoint / runtime

REF-HELM-PLATFORM

Kind: Referenced Element Name: Helm Runtime Platform

platform_service / runtime

REF-POSTGRES-DRIVER

Kind: Referenced Element Name: PostgreSQL Client Driver

third_party_library / code

REQ-PAY-001

Kind: Requirement Name: REQ-PAY-001 Resolves to: canonical While checkout is active, when payment authorization is requested, the payments system shall create a payment session and request authorization from bank gateway.

REQ-PAY-002

Kind: Requirement Name: REQ-PAY-002 Resolves to: canonical Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval.

REQ-PAY-003

Kind: Requirement Name: REQ-PAY-003 Resolves to: canonical If payment is declined, then the payments system shall present a decline reason and a retry option to customer.

REQ-PAY-004

Kind: Requirement Name: REQ-PAY-004 Resolves to: canonical While checkout is active, when payment authorization is requested and risk score is high, the payments system shall place the payment in review pending state and notify support agent.

REQ-PAY-005

Kind: Requirement Name: REQ-PAY-005 Resolves to: canonical Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id and risk score.

REQ-PAY-006

Kind: Requirement Name: REQ-PAY-006 Resolves to: canonical If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer.

11.2. Catalog References

ACT-CUSTOMER

Kind: Catalog Actor Name: customer Resolves to: canonical End user submitting payment details during checkout.

ACT-PLATFORM-OPERATOR

Kind: Catalog Actor Name: platform operator Resolves to: canonical Role responsible for GitOps and platform lifecycle operations.

ACT-SUPPORT

Kind: Catalog Actor Name: support agent Resolves to: canonical Operations role that investigates declines and high-risk review cases.

3DS required

Kind: Catalog Alias Name: 3DS verification is required Resolves to: canonical Alias of COND-3DS-REQUIRED

auth

Kind: Catalog Alias Name: payment authorization Resolves to: canonical Alias of FU-PAYMENT-AUTHORIZATION

bank API

Kind: Catalog Alias Name: bank gateway endpoint Resolves to: canonical Alias of REF-BANK-GATEWAY-ENDPOINT

bank link down

Kind: Catalog Alias Name: bank link is unavailable Resolves to: canonical Alias of EVT-BANK-LINK-UNAVAILABLE

bank link up

Kind: Catalog Alias Name: bank link is available Resolves to: canonical Alias of STATE-BANK-LINK-UP

buyer

Kind: Catalog Alias Name: customer Resolves to: canonical Alias of ACT-CUSTOMER

callback replay

Kind: Catalog Alias Name: replayed authorization callback Resolves to: canonical Alias of AV-REPLAYED-AUTH-CALLBACK

checkout

Kind: Catalog Alias Name: checkout handling Resolves to: canonical Alias of FU-CHECKOUT

checkout active

Kind: Catalog Alias Name: checkout is active Resolves to: canonical Alias of STATE-CHECKOUT-ACTIVE

day1 provisioning

Kind: Catalog Alias Name: cluster provisioning Resolves to: canonical Alias of FU-CLUSTER-PROVISIONING

day2 operations

Kind: Catalog Alias Name: gitops operations Resolves to: canonical Alias of FU-GITOPS-OPERATIONS

decline code

Kind: Catalog Alias Name: decline reason Resolves to: canonical Alias of DATA-DECLINE-REASON

dependency compromise

Kind: Catalog Alias Name: compromised dependency Resolves to: canonical Alias of AV-COMPROMISED-DEPENDENCY

fraud abuse

Kind: Catalog Alias Name: fraudulent transaction pattern Resolves to: canonical Alias of AV-FRAUDULENT-TRANSACTION-PATTERN

fraud check

Kind: Catalog Alias Name: fraud check is enabled Resolves to: canonical Alias of FEAT-FRAUD-CHECK

fraud detection system

Kind: Catalog Alias Name: fraud evaluation Resolves to: canonical Alias of FG-FRAUD

fraud domain

Kind: Catalog Alias Name: fraud evaluation Resolves to: canonical Alias of FG-FRAUD

fraud scoring

Kind: Catalog Alias Name: risk scoring Resolves to: canonical Alias of FU-RISK-SCORING

helm

Kind: Catalog Alias Name: helm platform Resolves to: canonical Alias of REF-HELM-PLATFORM

high risk

Kind: Catalog Alias Name: risk score is high Resolves to: canonical Alias of EVT-RISK-HIGH

malicious request

Kind: Catalog Alias Name: malicious api request Resolves to: canonical Alias of AV-MALICIOUS-API-REQUEST

manual review

Kind: Catalog Alias Name: support review Resolves to: canonical Alias of FU-SUPPORT-REVIEW

normal mode

Kind: Catalog Alias Name: normal mode is active Resolves to: canonical Alias of MODE-NORMAL

payment auth requested

Kind: Catalog Alias Name: payment authorization is requested Resolves to: canonical Alias of EVT-PAYMENT-AUTH-REQUESTED

payment declined

Kind: Catalog Alias Name: payment is declined Resolves to: canonical Alias of EVT-PAYMENT-DECLINED

payment fraud

Kind: Catalog Alias Name: fraudulent transaction pattern Resolves to: canonical Alias of AV-FRAUDULENT-TRANSACTION-PATTERN

payments domain

Kind: Catalog Alias Name: payments Resolves to: canonical Alias of FG-PAYMENTS

payments system

Kind: Catalog Alias Name: payments Resolves to: canonical Alias of FG-PAYMENTS

platform engineering

Kind: Catalog Alias Name: platform Resolves to: canonical Alias of FG-PLATFORM

platform ops

Kind: Catalog Alias Name: platform operator Resolves to: canonical Alias of ACT-PLATFORM-OPERATOR

postgres driver

Kind: Catalog Alias Name: postgresql client driver Resolves to: canonical Alias of REF-POSTGRES-DRIVER

review pending

Kind: Catalog Alias Name: payment review is pending Resolves to: canonical Alias of STATE-REVIEW-PENDING

risk above threshold

Kind: Catalog Alias Name: risk threshold is exceeded Resolves to: canonical Alias of COND-RISK-THRESHOLD-EXCEEDED

support

Kind: Catalog Alias Name: support agent Resolves to: canonical Alias of ACT-SUPPORT

support notified

Kind: Catalog Alias Name: support agent is notified Resolves to: canonical Alias of EVT-SUPPORT-NOTIFIED

AV-COMPROMISED-DEPENDENCY

Kind: Catalog Attack Vector Name: compromised dependency Resolves to: canonical Supply-chain compromise in runtime image or code dependency.

AV-FRAUDULENT-TRANSACTION-PATTERN

Kind: Catalog Attack Vector Name: fraudulent transaction pattern Resolves to: canonical Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals.

AV-MALICIOUS-API-REQUEST

Kind: Catalog Attack Vector Name: malicious api request Resolves to: canonical Crafted payload attempting to bypass validation or authorization logic.

AV-REPLAYED-AUTH-CALLBACK

Kind: Catalog Attack Vector Name: replayed authorization callback Resolves to: canonical Replay of a valid callback to induce duplicate or unauthorized state transitions.

COND-3DS-REQUIRED

Kind: Catalog Condition Name: 3DS verification is required Resolves to: canonical Condition indicating strong customer authentication must be performed.

COND-RISK-THRESHOLD-EXCEEDED

Kind: Catalog Condition Name: risk threshold is exceeded Resolves to: canonical Condition indicating risk score crossed a policy threshold for escalation.

DATA-DECLINE-REASON

Kind: Catalog Data Term Name: decline reason Resolves to: canonical Structured reason code or message explaining why authorization failed.

DATA-PAYMENT-ID

Kind: Catalog Data Term Name: payment id Resolves to: canonical Stable identifier used to correlate a payment across services and records.

DATA-RISK-SCORE

Kind: Catalog Data Term Name: risk score Resolves to: canonical Numeric fraud score produced by risk evaluation logic.

EVT-BANK-LINK-UNAVAILABLE

Kind: Catalog Event Name: bank link is unavailable Resolves to: canonical Event raised when payment engine cannot reach the external bank gateway.

EVT-PAYMENT-AUTH-REQUESTED

Kind: Catalog Event Name: payment authorization is requested Resolves to: canonical Event raised when checkout asks payment engine to authorize a transaction.

EVT-PAYMENT-DECLINED

Kind: Catalog Event Name: payment is declined Resolves to: canonical Event indicating authorization was rejected and customer decline handling must execute.

EVT-RISK-HIGH

Kind: Catalog Event Name: risk score is high Resolves to: canonical Event emitted when computed fraud risk exceeds the configured high-risk threshold.

EVT-SUPPORT-NOTIFIED

Kind: Catalog Event Name: support agent is notified Resolves to: canonical Event indicating manual review notification was sent to support operations.

FEAT-FRAUD-CHECK

Kind: Catalog Feature Name: fraud check is enabled Resolves to: canonical Feature flag controlling whether risk scoring is required before authorization.

FG-FRAUD

Kind: Catalog Functional Group Name: fraud evaluation Resolves to: canonical Functional group for risk scoring and fraud review capabilities.

FG-PAYMENTS

Kind: Catalog Functional Group Name: payments Resolves to: canonical Functional group for checkout and authorization capabilities.

FG-PLATFORM

Kind: Catalog Functional Group Name: platform Resolves to: canonical Functional group for provisioning and platform operations capabilities.

FU-CHECKOUT

Kind: Catalog Functional Unit Name: checkout handling Resolves to: canonical Handles checkout session start and customer-facing payment messaging.

FU-CLUSTER-PROVISIONING

Kind: Catalog Functional Unit Name: cluster provisioning Resolves to: canonical Provides cluster-level platform setup and environment bootstrap.

FU-GITOPS-OPERATIONS

Kind: Catalog Functional Unit Name: gitops operations Resolves to: canonical Operates declarative delivery and release management flows.

FU-PAYMENT-AUTHORIZATION

Kind: Catalog Functional Unit Name: payment authorization Resolves to: canonical Coordinates authorization flow and bank gateway interaction.

FU-RISK-SCORING

Kind: Catalog Functional Unit Name: risk scoring Resolves to: canonical Evaluates transaction risk and returns score classification.

FU-SUPPORT-REVIEW

Kind: Catalog Functional Unit Name: support review Resolves to: canonical Supports manual review and escalation handling for risky transactions.

MODE-NORMAL

Kind: Catalog Mode Name: normal mode is active Resolves to: canonical Standard operating mode with full external dependency usage enabled.

REF-BANK-GATEWAY-ENDPOINT

Kind: Catalog Referenced Element Name: bank gateway endpoint Resolves to: canonical External banking endpoint used for card authorization and decline decisions.

REF-HELM-PLATFORM

Kind: Catalog Referenced Element Name: helm platform Resolves to: canonical Runtime packaging and deployment substrate for workloads.

REF-POSTGRES-DRIVER

Kind: Catalog Referenced Element Name: postgresql client driver Resolves to: canonical Third-party library used for application data persistence connectivity.

STATE-BANK-LINK-UP

Kind: Catalog State Name: bank link is available Resolves to: canonical Connectivity state showing bank gateway communication is currently healthy.

STATE-CHECKOUT-ACTIVE

Kind: Catalog State Name: checkout is active Resolves to: canonical Processing state where the user is actively submitting or confirming payment.

STATE-REVIEW-PENDING

Kind: Catalog State Name: payment review is pending Resolves to: canonical Transaction state requiring manual support review before final disposition.

11.3. Inferred Runtime References

payments

Kind: Runtime cluster Owner: FU-CLUSTER-PROVISIONING Source: examples/payments-engineering-sample/infra/terraform/main.tf

flux-system/payments-sample-source

Kind: Runtime gitrepository Owner: FU-GITOPS-OPERATIONS Source: examples/payments-engineering-sample/infra/flux/sources/payments-source.yaml

payments/checkout-api

Kind: Runtime helmrelease Owner: FU-CHECKOUT Source: examples/payments-engineering-sample/infra/flux/releases/checkout-api.yaml

payments/payment-engine

Kind: Runtime helmrelease Owner: FU-PAYMENT-AUTHORIZATION Source: examples/payments-engineering-sample/infra/flux/releases/payment-engine.yaml

risk/risk-scorer

Kind: Runtime helmrelease Owner: FU-RISK-SCORING Source: examples/payments-engineering-sample/infra/flux/releases/risk-scorer.yaml

flux-system/payments-apps

Kind: Runtime kustomization Owner: FU-GITOPS-OPERATIONS Source: examples/payments-engineering-sample/infra/flux/kustomizations/payments-apps.yaml

flux_system

Kind: Runtime namespace Owner: FU-GITOPS-OPERATIONS Source: examples/payments-engineering-sample/infra/terraform/main.tf

payments

Kind: Runtime namespace Owner: FU-CHECKOUT Source: examples/payments-engineering-sample/infra/terraform/main.tf

risk

Kind: Runtime namespace Owner: FU-RISK-SCORING Source: examples/payments-engineering-sample/infra/terraform/main.tf

11.4. Inferred Code References

gopkg.in/yaml.v3

Kind: Code library_external Owner: FU-CHECKOUT Source: checkout_api.go

serde_json::json

Kind: Code library_external Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs

zod

Kind: Code library_external Owner: FU-RISK-SCORING Source: risk_scorer.ts

./support/audit_envelope

Kind: Code library_first_party Owner: FU-RISK-SCORING Source: risk_scorer.ts

crate::domain::events::PaymentEvent

Kind: Code library_first_party Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs

github.com/labeth/engineering-model-go/validate

Kind: Code library_first_party Owner: FU-CHECKOUT Source: checkout_api.go

fmt

Kind: Code library_stdlib Owner: FU-CHECKOUT Source: checkout_api.go

checkout_api.go

Kind: Code source_file Owner: FU-CHECKOUT Source: checkout_api.go

domain/events.rs

Kind: Code source_file Owner: FU-PAYMENT-AUTHORIZATION Source: domain/events.rs

payment_engine.rs

Kind: Code source_file Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs

risk_scorer.ts

Kind: Code source_file Owner: FU-RISK-SCORING Source: risk_scorer.ts

support/audit_envelope.ts

Kind: Code source_file Owner: FU-RISK-SCORING Source: support/audit_envelope.ts

CODE-AUTHORIZEPAYMENT

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:10

CODE-BUILDREVIEWREASON

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:49

CODE-CALCULATERISKSCORE

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:7

CODE-CLASSIFYRISK

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:20

CODE-CREATEAUDITRECORD

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:36

CODE-ENRICHAUDITMETADATA

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:44

CODE-HANDLEBANKLINKUNAVAILABLE

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:33

CODE-HANDLEBANKUNAVAILABLE

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:31

CODE-ISHIGHRISK

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:28

CODE-NOTIFYSUPPORT

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:51

CODE-PERSISTAUDITRECORD

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:59

CODE-PLACEINREVIEW

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:43

CODE-REQUESTBANKAUTHORIZATION

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:25

CODE-SHOWDECLINEREASON

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:36

CODE-SHOWRETRYOPTION

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:41

CODE-SHOWTEMPORARYUNAVAILABLE

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:46

CODE-STARTSESSION

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:15

CODE-SUBMITAUTHORIZATIONREQUEST

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:26

11.5. Verification References

VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597

Kind: Verification Check Name: Authorized Checkout Flow Resolves to: canonical Kind: e2e; Status: not-run; Evidence: tests/e2e/authorized_checkout_flow.yaml

VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2

Kind: Verification Check Name: Audit Record Persistence Test Resolves to: canonical Kind: integration; Status: partial; Evidence: test-results/audit-persistence-report.md, tests/integration/audit_record_persistence_test.ts

VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD

Kind: Verification Check Name: Fraud Gate Test Resolves to: canonical Kind: integration; Status: pass; Evidence: test-results/integration-fraud-gate.json, tests/integration/fraud_gate_test.rs

VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26

Kind: Verification Check Name: Contract Checkout Decline Resolves to: canonical Kind: test; Status: pass; Evidence: test-results/contract-checkout-decline.json